

CDE2605 UROP

AY 2024/2025 Semester 2



Robotic Automation of Reliability Testing for Biomedical Camera Probes

<u>Name</u>	<u>Student #</u>
Marcus Wong	A0273713E

Contents Page

1. List of figures	3
2. List of tables	3
3. Introduction	4
4. Literature Review	5
4.1 UR5e user tutorial	5
4.2 Analysis of the forces generated by the thumb	6
4.3 CAD user tutorial	6
5. Problem Statement Description	6
5.1 Assessing the probe	7
5.2 Understanding TDS' previous implementations	8
5.3 Working Environment	9
6. Methodology	10
6.1 Configuration of TCP	10
6.2 Waypoint calibration	10
6.3 Generating waypoints for circular movements	12
6.4 Programming the predefined testing actions	13
6.5 Measurement of force at the joystick in the joystick frame	14
6.6 Making the stand for the socket joint	18
6.7 Overall program	20
7. Evaluation, Limitations and Improvements	20
7.1 Waypoint calibration	20
7.2 Measurement of force at the joystick in the joystick frame	21
7.3 Making the stand for the socket joint	21
7.4 Other considerations	22
8. Conclusion	22
9. References	22
10. Appendix	23
Appendix A: Gantt Chart	23
Appendix B: Project Expenditure	24
Appendix C: Other CAD iterations of the stand for the socket joint	25
Appendix D: Source code	25

1. List of figures

Figure 1: Joystick controlled biomedical camera probe	4
Figure 2: Control tablet accessing the PolyScope interface, displaying URScript.....	5
Figure 3: TCP is the relative coordinate from the blue dot to the end effector	5
Figure 4: Biomedical probe with full tilt to the right/left (Above/below respectively)	7
Figure 5: Hard stops are the rectangular metal piece, each with two screws	7
Figure 6: Waypoint labelling of the hard stops.....	8
Figure 7: Initial position (left) and compressed (right) position of the ball & socket joint.....	8
Figure 8: Left and right displacement of the joystick.....	9
Figure 9: Enclosed setup of the UR5e	9
Figure 10: New TCP position	10
Figure 11: URScript for obtaining force in TCP frame	11
Figure 12: Diagram of waypoints, where more complex waypoints are indicated by 'x'.	11
Figure 13: Diagram of via point approximations	12
Figure 14: URScript of the pose transformation from C (Point_1_1)	12
Figure 15: Testing actions required	13
Figure 16: Motion of Twist Right.....	13
Figure 17: Partial URscript for popup interface, test motion control and number of tests	14
Figure 18: Force at joystick position in the joystick frame	14
Figure 19: Model of variables needed for force calculations	15
Figure 20: URScript for obtaining the forces applied at the joystick in the TCP frame	15
Figure 21: URScript for obtaining the angle of joystick deflection.....	15
Figure 22: Example of force validation	16
Figure 23: URScript carrying out force reading collection for the 'Down' direction	17
Figure 24: First iteration of the stand	18
Figure 25: Second iteration of the stand	19
Figure 26: 3D printed stand with the inductive sensor and socket joint	19
Figure 27: URScript to simulate retrieval of socket joint.....	20
Figure 28: Preliminary Design of the stand	25
Figure 29: Intermediate iteration of stand, between the iterations of figure 24 and 25	25

2. List of tables

Table 1: Force values for Up Direction	17
Table 2: Force values for Down Direction	17
Table 3: Force values for Left Direction	18
Table 4: Force values for Right Direction	18

3. Introduction

This UROP project was part of an ongoing collaboration between TDS Technology and Olympus, where TDS is developing an automated reliability testing system for Olympus' joystick controlled biomedical camera probes, as seen in Figure 1 below. The main goal of this UROP project is to assist in the development and implementation of this testing system. These probes are used in medical procedures and are inserted into the human body, making their reliability critical to patient safety. Manual testing was previously used but proved to be inefficient and labour-intensive, especially since every probe must be individually tested.

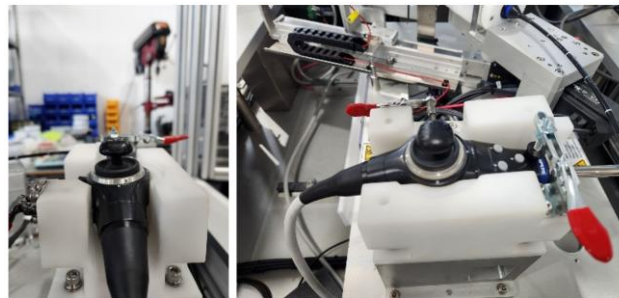


Figure 1: Joystick controlled biomedical camera probe

The company employed UR5e robots, as the primary platform for automating the testing process. To support this effort, the UR5e's interface and programming capabilities were studied to develop a comprehensive understanding of its operational functions. In parallel, a literature review was conducted on the biomechanics of the human thumb during joystick manipulation. In addition, proficiency in SOLIDWORKS was developed to support the mechanical design aspects of the project.

This project's primary objective was to automate four specific joystick movements of the biomedical probe, with an interface that allowed operators to select the type of movement and the number of repetitions. Additionally, this project also required the verification of the force applied to the joystick and the inclusion of a stand to securely hold the joystick's end effector after testing was completed. These objectives are described in the problem statement.

The typical project workflow involved weekly ideation and planning, during which code, workflow logic, and relevant calculations were developed. These plans were subsequently implemented and tested during scheduled site visits. Observations, errors, and areas for improvement were documented on-site to guide the next iteration of development. Regular updates were provided to both TDS personnel and UROP supervisor to ensure alignment with project goals, receive technical guidance on calculations, and obtain feedback for further improvement. The overall iterative progress and evaluation are documented in the methodology section.

Upon completion of the project's objectives, the workflow was demonstrated to both the TDS personnel and UROP supervisor and the necessary materials (i.e.: code and stand) were handed over. In addition, several key recommendations for improving the overall testing workflow were also provided to support continued development and optimisation of the system.

4. Literature Review

4.1 UR5e user tutorial

To achieve a strong foundational understanding of the UR5e robotic arm, including its operational capabilities and programming methodology, the official online training modules were completed, covering key aspects such as safety protocols, robot configuration, and motion programming using the PolyScope interface and URScript, the robot's proprietary scripting language. The robotic arm and its interface are primarily accessed through the Control Tablet, as shown in Figure 2 below.

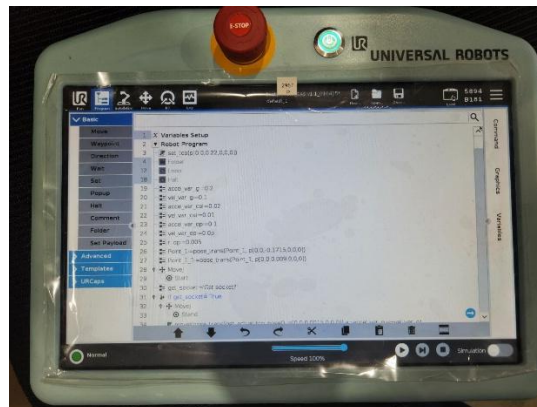


Figure 2: Control tablet accessing the PolyScope interface, displaying URScript

A core concept in the UR5e is the Tool Centre Point (TCP). It is the reference point on the end effector which the UR5e uses to execute movement commands. The robot moves this point during these commands, making accurate TCP calibration essential for precise task execution. An illustration of the TCP is shown in Figure 3.

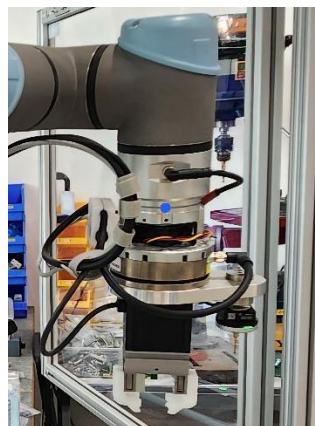


Figure 3: TCP is the relative coordinate from the blue dot to the end effector

Another closely related concept are the base frame and TCP frame. The base frame is the global coordinate system attached to the robot base, while the TCP frame moves with the tool, defining its local position and orientation. Understanding these frames is crucial for effective motion planning and pose control [1].

Compared to the standard block-based programming interface of the UR5e, URScript provides more flexible and powerful set of functions that enable computation and precise control over the robot's pose and motion. Two fundamental pose manipulation functions are `pose_trans` and `pose_inv`:

- `pose_trans(p1, p2)`: This function applies pose `p2` as an offset to pose `p1` and is primarily used to calculate new poses relative to a known frame. It is primarily used to calculate new target positions based on relative poses.
- `pose_inv(p)`: This function computes the inverse of a pose, effectively reversing the translation and rotation components. This function is useful for transforming coordinates between the tool and base frame.

These functions, together with motion commands like `movec` (circular motion), and `movep` (polynomial blended motion) are essential in path planning for the UR5e [2].

4.2 Analysis of the forces generated by the thumb

As the project aims to automate joystick manipulation using a robotic arm, it was necessary to first understand the biomechanics of the human thumb, particularly its force generation capabilities and modes of interaction with a joystick.

In the context of joystick operation, normal forces refer to forces exerted perpendicular to the joystick surface, which are essential for maintaining firm and consistent contact between the thumb and the joystick. These forces help stabilize the thumb during both static holding and dynamic manipulation. In contrast, tangential forces, which act parallel to the joystick surface, are responsible for displacing or moving the joystick in different directions.

A review of existing biomechanical literature indicates that the force output of the human thumb varies based on task type, direction of motion, and type of grip. During pressing tasks, the thumb can exert substantial force, with downward forces of up to 100N [3]. While these values do not directly represent the forces exerted when operating this probe's joystick, they serve as a reasonable upper bound for estimating the expected force range. In practice, the required operational force should be significantly lower to prevent overexertion of the thumb, especially given the repetitive nature of joystick usage and the fine motor control required for precise movements. This is particularly important in the medical context of the device, where excessive resistance could compromise usability, user comfort, and long-term operability.

4.3 CAD user tutorial

To support the mechanical aspects of the project—specifically the design and fabrication of the stand—a structured self-directed learning approach was adopted using SOLIDWORKS, a professional-grade parametric CAD software. Official tutorials and user documentation were utilized to develop proficiency in essential design functions and to gain familiarity with the broader capabilities of CAD, including 2D sketching and 3D extrusion.

5. Problem Statement Description

The primary objective of this project was to design, implement, and refine an automated testing workflow that extended and improved upon the existing progress made by TDS Technology. This project is intended to fulfil several key functional requirements:

- Reliably execute a set of predefined joystick movements using the UR5e robot.
- Capture the forces exerted by the UR5e during these joystick movements.
- Provide a user interface allowing the operator to select specific test sequences and define the number of repetitions.
- Design a stand and corresponding workflow to retrieve and return the end effector before and after testing.

These functional requirements will be further discussed in the “Methodology” section below. To develop a viable solution, it was first necessary to understand the mechanical behaviour of the joystick component within the biomedical probe.

5.1 Assessing the probe

The joystick is operated via the user’s thumb and controls a camera at the end of the probe, allowing for articulation within a half-spherical workspace. The maximum allowable tilt in any direction is constrained to 90 degrees. This is shown in Figure 4 below:

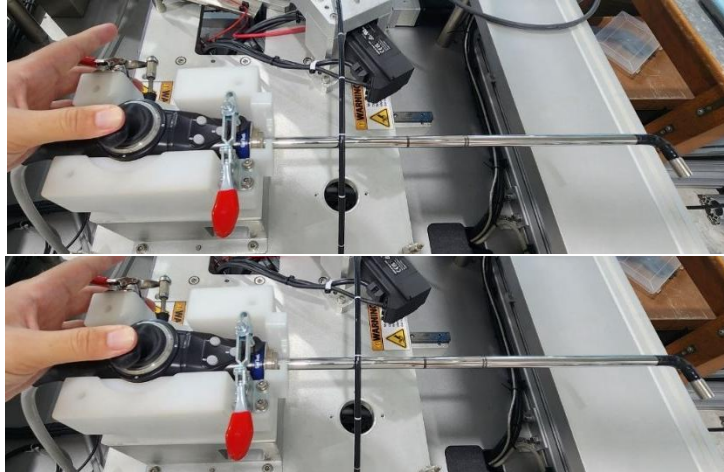


Figure 4: Biomedical probe with full tilt to the right/left (Above/below respectively)

A significant complication arises from variability introduced during the probe’s manufacturing process, where the translation of the joystick results in slight differences in the resulting motion at the end of the probe. To enforce the 90-degree tilt constraint, each probe is fitted with mechanical stoppers that are manually installed by workers, as shown in Figure 5. This leads to varying translational requirements and different force profiles for the UR5e to accurately replicate the intended joystick motions for different probes.



Figure 5: Hard stops are the rectangular metal piece, each with two screws

As a result, a calibration procedure was deemed necessary for each individual probe prior to testing. This step ensured that the UR5e could adjust its motion parameters to match the physical constraints of the specific probe being tested.

Furthermore, the physical shape of the joystick’s hard stop was not of a standard shape as seen above. This presented a challenge in generating waypoints during calibration for the motion path. A simple circular trajectory would either fail to reach the full 90-degree tilt (if undersized) or risk damaging the probe (if oversized). Hence, the minimal number of waypoints required for full trajectory of the joystick is listed in Figure 6, labelled by numbers. (Note: The diagram does not depict the actual spatial locations of the waypoints but rather indicates the joystick’s position when the UR5e moves to each respective waypoint.)

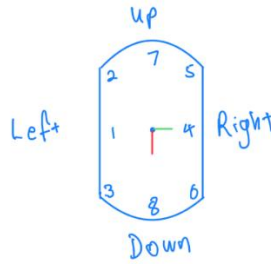


Figure 6: Waypoint labelling of the hard stops

5.2 Understanding TDS' previous implementations

First Approach

TDS's initial approach involved directly attaching the end effector of the UR5e robotic arm to the joystick interface of the biomedical probe. The robot was programmed to move around the joystick's pivot point to replicate the full range of required test movements with precision.

Upon reviewing the implementation and corresponding code, it became evident that this method was both labour-intensive and difficult to generalize across probes. For the UR5e, while the type of motion between waypoints can be specified, the exact trajectory is internally computed by the robot's control software. This limits fine-grained control over the motion path, particularly when navigating complex or non-uniform motion regions. As a result, when only a minimal number of waypoints were defined, the robot's path deviated from the intended trajectory, often drifting toward the centre of the probe rather than closely following the desired motion.

To achieve more accurate trajectories, additional intermediate waypoints were introduced. These reduced the interpolation distance between consecutive points, thereby improving path fidelity. However, this workaround significantly increased the programming complexity, as each waypoint had to be manually calculated and initialised.

Second Approach

TDS decided to adopt a simplified approach to control the motion of the UR5e by incorporating a ball and socket joint mechanism. In this configuration, an additional arm was attached to the joystick end of the biomedical probe, terminating in a spherical ball. The end effector of the UR5e was equipped with a corresponding socket mechanism, capable of vertical displacement through a spring-loaded design, as shown in Figure 7.

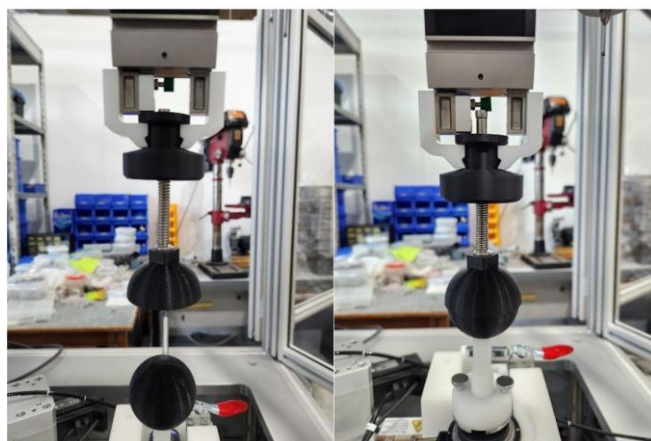


Figure 7: Initial position (left) and compressed (right) position of the ball & socket joint

During operation, the UR5e first descends vertically, pressing the socket onto the ball joint, thereby compressing the spring and establishing the initial starting position. Subsequent XY-plane translations of the UR5e allow the spring to decompress, which displaces the joystick extension at an angle, as shown in Figure 8. This interaction effectively replicates joystick rotations through controlled XY planar motions of the UR5e.

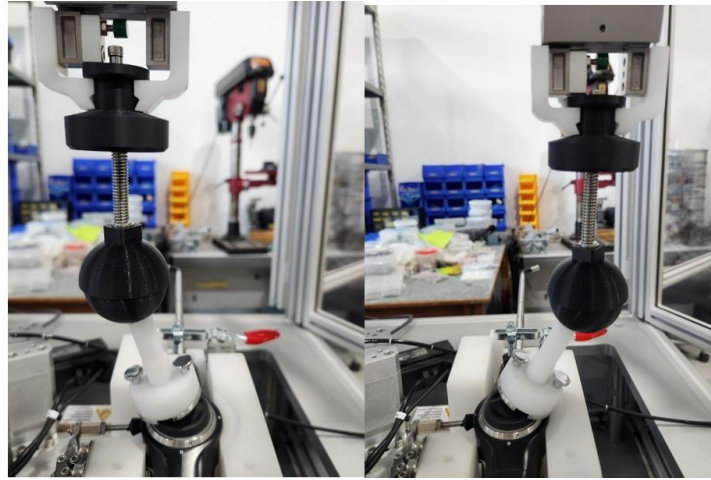


Figure 8: Left and right displacement of the joystick

This design simplified the robot's motion control, as it constrained movement to only the XY plane. At this point, TDS had a partial implementation of waypoint calibration, waypoint generation, and a basic test movement. The task moving forward was to review the existing codebase and mechanical design, identify areas for improvement, and continue development to fulfil the key functional requirements outlined earlier.

5.3 Working Environment

The entire testing procedure was conducted within a shared enclosed setup, which is utilized by multiple ongoing projects involving the same UR5e robotic arm. This setup is shown in Figure 9. This environment is centrally managed by a PLC (Programmable Logic Controller) that governs the operation of the end effector—specifically the white gripping component shown in Figure 7—which is responsible for securing and releasing the socket joint. Notably, the UR5e is mounted in an inverted orientation within the enclosure, resulting in a reversal of its Z-axis direction (i.e., positive Z points downward, and negative Z points upward). For spatial reference of the UR5e and joystick, the positive Y and X directions align with the red and green axes, respectively, as illustrated in Figure 6.



Figure 9: Enclosed setup of the UR5e

6. Methodology

6.1 Configuration of TCP

For accurate motion of the UR5e robotic arm, it was necessary to ensure that the TCP was configured correctly. The UR5e's internal control scripts rely on the TCP for inverse kinematics calculations, determining how the robot moves from one waypoint to another. Upon reviewing the initial implementation, it was found that the TCP had been incorrectly defined at the pivot point of the joystick. While the robot was still able to perform movements, this setup was not technically correct. The TCP should ideally follow the motion of the tool flange, as this ensures consistency between physical tool behaviour and programmed motion. An incorrectly set TCP can result in unpredictable movements when programming additional movement commands and can also lead to inaccurate force measurements.

The corrected TCP position was moved along the Z-axis—from **0.3915m** to **0.22m** from the tool flange, as shown in Figure 10. Consequently, “Point_1”, which represents the resting and initially compressed state of the ball-and-socket joint, required a compensatory offset of **-0.1715m** along the Z-axis. This adjustment was critical, as “Point_1” serves as a reference for calculating subsequent waypoints.



Figure 10: New TCP position

6.2 Waypoint calibration

As previously discussed, fixed waypoint positions could not be hardcoded due to variability in the physical setup. To address this, a calibration procedure was developed to dynamically determine waypoint positions using force feedback. The core principle of this method is that, as the robot approaches a target position, contact with the joystick will cause a rapid increase in measured force. Once a predefined force threshold is exceeded—without risking damage to the joystick—the corresponding position is recorded as the waypoint. To implement this approach, it is first necessary to acquire force readings expressed in the TCP frame at the TCP position. However, the UR5e only provides force readings in the base frame. The following section outlines the procedure used to obtain the required force measurements [4].

```

F_base:=get_tcp_force()
Fx_base:=F_base[0]
Fy_base:=F_base[1]
Fz_base:=F_base[2]
tcp:=get_actual_tcp_pose()
tcp_base:=p[tcp[0],tcp[1],tcp[2],0,0,0]
F_tool:=pose_trans(pose_inv(tcp), pose_trans(tcp_base, p[Fx_base,Fy_base,Fz_base,0,0,0]))
Fx_tool:=F_tool[0]
Fy_tool:=F_tool[1]
Fz_tool:=F_tool[2]

```

Figure 11: URScript for obtaining force in TCP frame

From Figure 11, the variable `F_base` captures the force vector measured at the TCP, expressed in the base frame. This vector includes both linear force components and torques. As the testing motions do not apply rotational movements on the joystick, only the linear force components (`Fx_base`, `Fy_base`, and `Fz_base`) are extracted for further processing.

Subsequently, the current TCP pose is retrieved, which returns the position and orientation of the TCP relative to the base frame. To isolate translational motion, `tcp_base` is constructed using the TCP's position while setting the rotational components to zero.

The linear force vector is then represented as a pose with no rotation. This pose is transformed, effectively relocating the force vector to the TCP's position, still expressed in the base frame. To convert this into the TCP frame, the inverse of the TCP pose is applied using `pose_inv(tcp)`. This yields a transformation from the base frame into the TCP frame. The final transformation provides the equivalent force vector expressed in the TCP coordinate frame.

A separate thread is initiated for the above code to continuously output force readings in real-time while the main program executes the calibration routine. During calibration, the UR5e begins at the initial position and applies force along the X or Y axis and moves strictly along that axis until a predefined force threshold is reached. Once this threshold is met, the robot pauses briefly to allow the position to stabilise, after which the corresponding waypoint is recorded. This method is used to determine the positions of **waypoints 1, 4, 7, and 8**, which are shown in Figure 12 below.

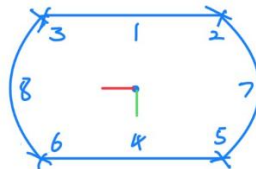


Figure 12: Diagram of waypoints, where more complex waypoints are indicated by 'x'.

For **waypoints 2, 3, 5, and 6**, the calibration process is more complex, as these positions are the intersection points between the straight paths and curved segments. For instance, after recording **waypoint 1**, the robot proceeds to determine **waypoint 2** by applying a negative Y-axis force, accompanied by a smaller, constant negative X-axis force. The latter serves only to keep the joystick aligned with the straight path from waypoint 1 to 2, while the main Y-axis force drives the joystick toward waypoint 2. Once a defined threshold in the negative Y-axis force is reached, which occurs due to the physical resistance from the corner where the straight path meets the arc—the robot pauses and records the position as waypoint 2. A similar approach is used to obtain waypoints 3, 5, and 6.

Following the earlier adjustment of the TCP position, the force thresholds used during calibration had to be retuned. Additionally, there was no validation that the joystick achieved maximum displacement. To address this, a structured workflow was developed to test and

refine the calibration process. The UR5e was first programmed to move to each waypoint. Then, the joystick was manually held in place while the robot was simultaneously lifted vertically to assess whether any slack was present at the contact point. If slack was detected, it indicated that the joystick did not reach the hard stop, prompting an increase in the corresponding force threshold. This adjustment was further validated through visual inspection, by confirming a 90-degree tilt at the end of the probe when proper contact was made. This calibration process was conducted iteratively using a trial-and-error approach to ensure consistent and accurate waypoint acquisition. A full demonstration of the calibration process can be found [here](#).

6.3 Generating waypoints for circular movements

One of the key challenges in implementing the predefined joystick movements involve generating accurate circular trajectories, such as from waypoint 3 to 8. For the UR5e to execute circular motions, an intermediate arc waypoint must be provided in addition to the start and end points. However, obtaining this intermediate point accurately during the calibration process was not feasible. As a result, approximations were employed to estimate the intermediate arc waypoint, allowing the UR5e to approximate the desired circular trajectory.

Attempt 1

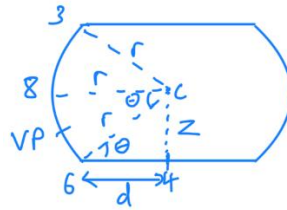


Figure 13: Diagram of via point approximations

Referring to the Figure 13 above, an assumption was made that the path through waypoints 3, 8, and 6 forms an arc of a circle, with a common radius from the centre point C. The radius, r , can be conveniently determined by measuring the Y-axis distance between waypoint 8 and the centre C. Additionally, the Y-axis distance, d between waypoints 4 and 2 can be readily obtained. To obtain the angle theta, the formula below was used:

$$\theta = \cos^{-1}\left(\frac{d}{r}\right)$$

To determine the viapoint (VP), first apply the alternate angle rule and then divide theta by 2. Then, perform a pose transformation from C using the calculated angle and radius to obtain the coordinates of the VP. This is done in URScript as shown in Figure 14 below.

```
var_11:=pose_trans(Point_1_1, p[0,0,0,0,0,-angle_6_8/2])
var_12:=pose_trans(var_11,p[0,-radius_3_8,0,0,0,0])
var_13:=pose_trans(var_12, p[0,0,0,0,0,angle_6_8/2])
```

Figure 14: URScript of the pose transformation from C (Point_1_1)

The pose of point C is first rotated by $-\frac{\theta}{2}$, followed by a translation by the radius r . This yields the position of the VP, but with an incorrect orientation. To correct this, the pose is then rotated back by $\frac{\theta}{2}$ to restore the proper orientation. This procedure was repeated 3 more times, with their respective rotations and translations to obtain the VPs of all the arc movements.

Attempt 2

Due to the slight slack in the movements along the arc, an alternative method to obtain θ was trialled to improve VP estimation. Referring to figure 11:

$$\theta = \tan^{-1}\left(\frac{Z}{r}\right)$$

The calculation of θ now no longer relied on r . Then, the procedure for obtaining the VP remained largely the same as before. The key idea was that inaccuracies in the arc's radius would be most significant at the endpoints—waypoint 3 or 6—leading to errors in calculating θ . In contrast, the error of r at the VP was expected to be less pronounced. However, testing revealed no noticeable improvement in accuracy using this method. As a result, the calculations from *Attempt 1* were retained for the remainder of the project.

6.4 Programming the predefined testing actions

With all necessary waypoints successfully calculated and calibrated, the testing actions could then be programmed. The figure below illustrates the four required actions. In Figure 15 below, the arrows indicate the direction of joystick movement, while the numbers associated with each arrow represent the sequence in which the actions are performed.

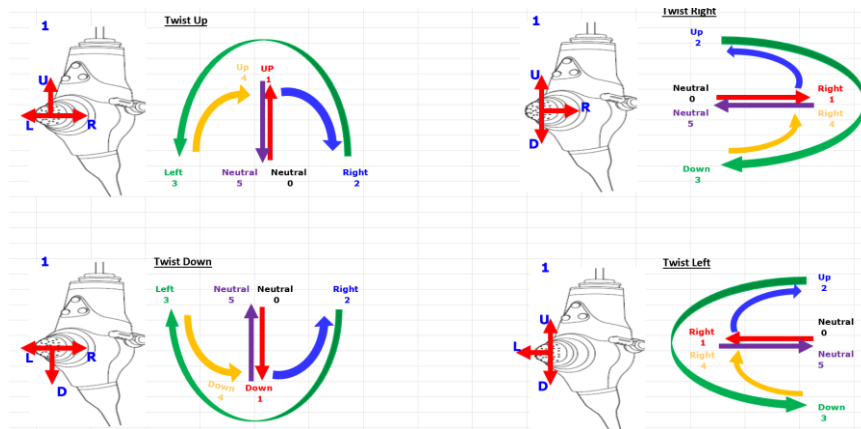


Figure 15: Testing actions required

```
Twist Right
movep(Limit_4,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_5,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_10,Limit_7,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movec(var_10,Limit_5,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_6,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_13,Limit_8,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movec(var_13,Limit_6,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_4,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)
```

Figure 16: Motion of Twist Right

The programmed actions consisted of executing movep and movec commands from the appropriate waypoints. Each action was designed to begin and end at the initial resting position of the joystick. movep was used for all linear motions, while movec was employed for arc movements, utilizing the previously calculated viapoints to define the curved paths. To ensure smooth and safe transitions, a stopj command was inserted at points where there were abrupt changes in direction, such as transitioning directly from waypoint 2 to 3 in the "Twist Right" motion, as illustrated in the figure above. The UR5e is not capable of handling such sudden directional shifts seamlessly, and the stopj command ensures that the robot comes to a controlled stop. To ensure consistency and reliability, each testing action was repeated five times under close observation. This was done to verify that all movements executed as

intended and to confirm that no mechanical issues or unintended behaviours occurred during operation. The general URScript workflow is shown in Figure 16 above.

These commands were subsequently placed within a loop structure and controlled using conditional variables, as shown in Figure 17. For each new probe tested, a pop-up interface appears on the UR5e control tablet, prompting the operator to select the desired test movements and specify the number of repetitions. This approach provides finer-grained control over the testing process and ensures that the correct number of repetitions is performed for each movement sequence. A demonstration of 1 loop of all the test movements can be found [here](#).

```

TL:='TL'
TR:='TR'
TU:='TU'
TD:='TD'
Test_Force:='Test force'
num_loops:='Number of repetitions?'
Loop num_loops>0
  If TL
    Twist Left
    movep(Limit_1,a=accel_var_op,v=vel_var_op,r=r_op)
    movep(Limit_2,a=accel_var_op,v=vel_var_op,r=r_op)

```

Figure 17: Partial URscript for popup interface, test motion control and number of tests

6.5 Measurement of force at the joystick in the joystick frame

The force data required by TDS and Olympus pertains specifically to the forces acting on the joystick itself. They only require the forces in four directions—Up, Down, Left, and Right—in the joystick frame. As such, it is necessary to transform the force data from the TCP frame into the joystick frame, as shown in Figure 18 below.

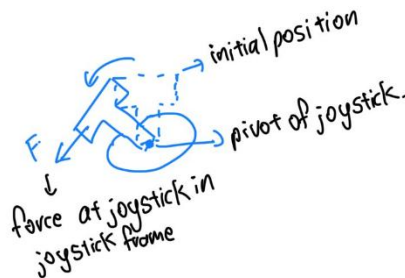


Figure 18: Force at joystick position in the joystick frame

To obtain the forces in the joystick frame from those measured in the TCP frame, a procedure similar to the one described in *Waypoint calibration* is followed. This involves first determining the forces at the joystick's position expressed in the TCP frame, followed by transforming these forces into the joystick's local coordinate frame. Due to the absence of a direct transformation between the TCP and joystick frames, manual calculations were required. These calculations were based on the following assumptions to simplify the model [6], as shown in Figure 19:

- The spring mechanism is sufficiently stiff, resulting in negligible force along the Z-axis being transmitted to the ball-and-socket joint and, by extension, the joystick.
- The ball-and-socket joint exhibits minimal flex, implying no significant mechanical losses through deformation.
- Friction within the ball-and-socket joint is negligible.
- There is no slack or backlash present in the joint assembly.
- The arm connecting the ball-and-socket joint to the joystick is perfectly rigid.

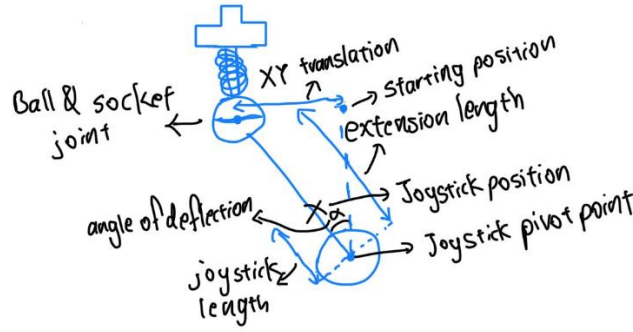


Figure 19: Model of variables needed for force calculations

Based on the assumptions outlined earlier, the force applied at the TCP can be reasonably approximated as being directly transmitted to the ball and socket joint. The next step is to determine the force at the joystick tip, rather than at the ball and socket joint.

By treating the pivot of the joystick as the system's effective rotational axis, the moment (torque) balance can be used to compute the force at the joystick tip in the TCP frame. Although the applied forces are not strictly perpendicular to the axis of rotation, the analysis remains valid because the ball and socket joint and the joystick tip lie along lines that form the same angle with respect to the pivot. Therefore, the ratio of their moment arms can be used to scale the forces accordingly. To accomplish this, the distance from the pivot to the ball and socket joint (extension length) and from the pivot to the joystick tip (joystick_length) needs to be measured. The formula below was applied, where its application in URScript is shown in Figure 20.

$$F_{x/y/z} = F_{TCP_{x/y/z}} * \left(\frac{extension_{length}}{joystick_{length}} \right)$$

```
extension_length:=0.092
joystick_length:=0.025
Fx_Joystick:=Fx_tool*(extension_length/joystick_length)
Fy_Joystick:=Fy_tool*(extension_length/joystick_length)
Fz_Joystick:=Fz_tool*(extension_length/joystick_length)
```

Figure 20: URScript for obtaining the forces applied at the joystick in the TCP frame

The next step involves converting the calculated forces into the joystick's coordinate frame. To achieve this, it is necessary to determine the joystick's angle of deflection. This can be done by calculating the straight-line distance between the initial ball and socket joint's reference position (Point_1_1) and the current TCP position. Using this displacement, the following formula is applied to compute the resultant deflection angle of the joystick, where its application in URScript is shown in Figure 21.

$$\alpha = \sin^{-1} \left(\frac{XY_Trans}{extension_{length}} \right)$$

```
XY_Trans:=point_dist(tcp, Point_1_1)
If XY_Trans>extension_length
  XY_Trans:=extension_length
Angle_Joystick:=r2d(asin(XY_Trans/extension_length))
```

Figure 21: URScript for obtaining the angle of joystick deflection

The cosine of the deflection angle can then be used to project the force onto the joystick's axis, allowing us to obtain the component of the force in the joystick frame, as described by the formula below.

$$\cos \alpha = F_{Joystick_Frame} / F_{TCP_Frame} \rightarrow F_{Joystick_Frame} = F_{TCP_Frame} * \cos \alpha$$

where F_{TCP_Frame} and $F_{Joystick_Frame}$ represent linear X and Y forces at the joystick position in their respective frames.

To ensure the validity of the computed force values, an industrial grade force sensor provided by TDS was used for experimental validation. Ideally, the validation would be performed directly at the joystick tip. However, due to the small size of the joystick and its orientation within the joystick frame, accurately positioning and applying the force sensor at that location proved to be impractical.

As a result, the validation was instead conducted at the ball and socket joint, as shown in Figure 22. This offered a larger and more accessible contact surface. Additionally, since this location lies within the TCP frame, the angle of force application is more straightforward and less susceptible to error.



Figure 22: Example of force validation

Given that the force computation relies on directly measurable quantities—such as the extension length and joystick length—and that the deflection angle is obtained from the UR5e's internal pose estimation (which is assumed to be highly accurate), the transformation from the ball and socket joint to the joystick is considered reliable. Therefore, if the measured force at the ball and socket joint matches the computed force values within the TCP frame, it can be reasonably concluded that the derived forces at the joystick tip are also accurate.

To streamline the collection of multiple data points required for force calculations—including joystick deflection angle, force components (X, Y, Z) at the ball-and-socket joint in the TCP frame, and corresponding forces at the joystick in the TCP frame—a structured testing workflow was implemented. Upon selecting the testing mode, the user is prompted to specify the direction to be tested (Up, Down, Left, or Right). The workflow then begins by repositioning the ball-and-socket joint to its initial resting state before proceeding with the test sequence.

Once the joint is reset, the robot moves to the predefined waypoint corresponding to the selected direction and pauses briefly to ensure the force readings are stable. During this pause, force pop-up prompts are displayed in a predefined sequence, allowing for consistent and efficient data collection, as shown in Figure 23 below. This structured approach eliminates the need to restart the program for each individual test, streamlining the data acquisition process.

```

If TestDown
  Test Down
  movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)
  stopj(d2r(80))
  movep(Limit_8,a=accel_var_op,v=vel_var_op,r=r_op)
  stopj(d2r(80))
  Wait: 1.0
  ForcePopUps
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup

```

Figure 23: URScript carrying out force reading collection for the 'Down' direction

Force data collection and verification were performed three times for each direction to ensure consistency and repeatability. Values greater than one were recorded with a precision of two decimal places, while values lesser than one were recorded to three significant figures. Since force was applied along a single axis in each test direction, only the corresponding axial force component was considered relevant and recorded. The TCP force values may be negative, depending on the direction in which the UR5e applies force. When evaluating the error between the TCP force and the verification force, and the joystick force in the joystick frame, only the magnitude is considered. The measured force values are all in Newton values and are summarized in the tables below.

Up (Negative Y Direction)

Trial	TCP Force	Verification Force	Joystick Force (TCP Frame)	Angle	Joystick Force (Joystick Frame)
1	-7.29	7.06	-26.90	35.60	21.87
2	-7.29	7.28	-24.20	35.60	19.68
3	-7.64	7.54	-26.73	35.59	21.74
Average	-7.41	7.29	-	-	21.10

Table 1: Force values for Up Direction

Average Validation Error: 1.65%

Down (Positive Y Direction)

Trial	TCP Force	Verification Force	Joystick Force (TCP Frame)	Angle	Joystick Force (Joystick Frame)
1	8.83	8.55	30.09	34.59	24.77
2	8.54	8.83	31.56	34.58	25.98
3	8.74	8.96	31.31	34.59	25.78
Average	8.70	8.78	-	-	25.51

Table 2: Force values for Down Direction

Average Validation Error: 3.00%

Left (Negative X Direction)

Trial	TCP Force	Verification Force	Joystick Force (TCP Frame)	Angle	Joystick Force (Joystick Frame)
1	-5.30	5.18	-18.20	28.6	15.98
2	-5.00	5.41	-16.01	28.6	14.06
3	-5.10	5.16	-16.75	28.6	14.71
Average	-5.13	5.25	-	-	14.92

Table 3: Force values for Left Direction

Average Validation Error: 2.29%

Right (Positive X Direction)

Trial	TCP Force	Verification Force	Joystick Force (TCP Frame)	Angle	Joystick Force (Joystick Frame)
1	4.94	5.25	16.38	27.69	14.50
2	5.23	5.03	17.68	27.68	15.66
3	5.01	5.21	17.42	27.68	15.43
Average	5.06	5.16	-	-	15.20

Table 4: Force values for Right Direction

Average Validation Error: 1.94%

The UR5e's built-in force sensor has a full-scale range of $\pm 50\text{N}$ with a specified accuracy of $\pm 4\text{ N}$, corresponding to a full-scale accuracy of approximately 8% [5]. However, in this project, all measured TCP forces fall below 10N, as seen from the tables above, under the columns labelled 'TCP Force'. At this low force range, the fixed $\pm 4\text{N}$ accuracy introduces a proportionally larger error. This is especially true for very low force measurements, such as those seen in tables 3 and 4, which could raise concerns about the reliability of low-force measurements.

The average validation errors across all tested directions, however, were found to be below 5%, indicating a strong degree of consistency between the UR5e's internal measurements and the validation results. This suggests that, although the UR5e's sensor is not optimised for high-precision measurements at low forces, it still provides sufficiently accurate readings for the purpose of this application.

6.6 Making the stand for the socket joint

In addition to the testing workflow and force validation, an additional workflow had to be developed to support a shared testing environment. This allowed the UR5e robotic arm to seamlessly switch between different tasks. To facilitate this, a dedicated stand was designed to securely hold the socket joint when not in use. This task was requested by TDS on top of the original project requirements.

The stand was designed to fit within the constraints of the enclosed workspace. It needs to interface with a PLC-controlled end effector, which is responsible for gripping and releasing the socket joint. (Note*: The UR5e does not have direct control over the end effector's gripping mechanism.) To detect whether the socket joint has been properly returned to the stand, an inductive sensor needs to be integrated into the PLC system. Figure 24 below illustrates the first iteration of the stand design.

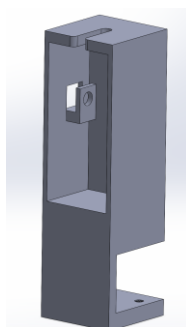


Figure 24: First iteration of the stand

The top section of the stand incorporates an elongated cutout designed, allowing the UR5e to slide the socket joint into place and rest it securely. An inductive sensor is mounted onto a hole-shaped extrusion on the side of the stand, which enables detection of the spring mechanism, thereby confirming the presence of the socket joint.

The base of the stand includes an L-shaped cutout intended to align with the mounting surface, as well as a threaded hole for a plunger screw that secures the stand firmly to the table. The initial design was found to be overly large, wasting material and not suitable for fabrication using smaller 3D printers. To address this, the stand was slightly redesigned to reduce its overall size, as shown in Figure 25, thereby lowering both the printing time and material costs.

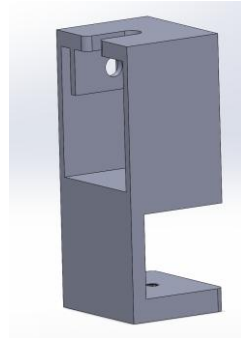


Figure 25: Second iteration of the stand

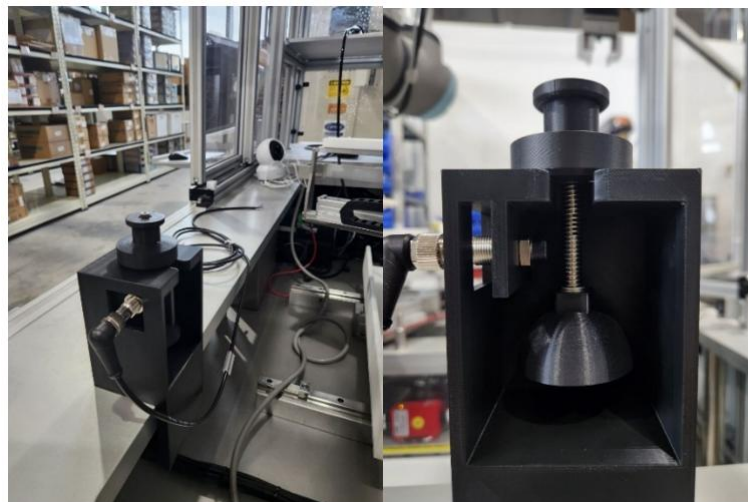


Figure 26: 3D printed stand with the inductive sensor and socket joint

The holder for the inductive sensor is designed with adjustable spacing, allowing the sensor to be positioned closer to or further from the spring to optimize joint detection. This flexibility is supported by the 8 mm sensing range of the selected inductive sensor, providing ample tolerance for fine-tuning. This is shown in the right image of Figure 26.

The image on the left of Figure 26 illustrates the stand installed within the enclosed workspace. It is positioned flush against the workbench, ensuring a safe distance from both the safety doors and the moving table within the workspace. A right-angled wire connector was used to interface the inductive sensor with the PLC controller. The cleaner cable management minimizes the risk of collisions with surrounding components.

For this project, direct access to the PLC controller was not available. As a result, synchronization between the inductive sensor, UR5e movements, and the end effector's opening or closing could not be implemented. To simulate the intended workflow, the UR5e

operations were carried out using pop-up prompts, as shown in Figure 27. These required the operator to visually confirm the socket joint's position before manually actuating the end effector and proceeding with the robot's next movement.

```
If get_socket2 True
  MoveJ
  Stand
  move1(pose_trans(get_actual_tcp_pose(), p[0,0,0.0015,0,0,0]),a=accel_var_g,v=vel_var_g)
  move1(pose_trans(get_actual_tcp_pose(), p[0,-0.044,0,0,0,0]),a=accel_var_g,v=vel_var_g)
  var_14:='Socket Gripped?'
  move1(pose_trans(get_actual_tcp_pose(), p[0,0.05,0,0,0,0]),a=accel_var_g,v=vel_var_g)
```

Figure 27: URscript to simulate retrieval of socket joint

Testing showed that the UR5e was able to reliably retrieve the socket joint from the stand. A demonstration of the placing and retrieval process can be found [here](#). As shown, it encountered difficulty when placing the socket joint back onto the stand. This issue arose due to the limited opening range of the end effector, which caused the socket joint to be unintentionally dragged along as the UR5e retracted. To address this issue, the UR5e was programmed to move further into the stand after the end effector was opened to release the socket joint. This ensured that the socket joint rested securely against the stopper, preventing it from being inadvertently displaced during the robot's retraction. A demonstration can be viewed [here](#). However, this was not implemented in the final iteration, due to concerns that it could hit the enclosure door.

6.7 Overall program

The overall program workflow of the UR5e can be found [here](#).

7. Evaluation, Limitations and Improvements

7.1 Waypoint calibration

During the calibration procedure for linear joystick motions, achieving an appropriate force in the X-direction proved challenging. The goal was to maintain the joystick on its linear path without applying excessive force that could damage the hard stop. However, during testing, it became evident that the applied force during calibration was too high. This was observed when the joystick was manually manipulated post-calibration, revealing a rough and uneven linear path, indicative of wear or indentation caused by repeated forceful contact with the hard stop. This compromises the integrity of the probe reducing its lifespan. Conversely, applying insufficient force during calibration led to excessive slack in the recorded motion path, compromising repeatability and the accuracy of subsequent automated tests.

To address this issue, an improved calibration procedure is proposed. Instead of relying solely on force-based detection of waypoints, the operator can manually guide the joystick to waypoints 2, 3, 5, and 6. This approach eliminates ambiguity in positioning and ensures precise, damage-free calibration. It can be extended to include via-points as well, allowing the entire trajectory to be defined with high accuracy.

Since the UR5e does not support Freedrive mode (Moving the UR5e manually) via URScript, this improved workflow would require an alternative method for manual guidance. One viable solution involves the robot's digital input output system. A digital output signal can be programmed within the URScript to trigger a digital input on the controller, which is configured to activate Freedrive mode. This would allow the operator to safely move the robot manually during calibration, with URScript pop-ups prompting him at each step of the procedure.

7.2 Measurement of force at the joystick in the joystick frame

The internal force sensor of the UR5e lacks the precision required for tasks involving small, fine-grained forces, such as those encountered in this project. To mitigate this limitation, an industrial-grade external force sensor was installed on the UR5e unit used in this project. However, access to this sensor was restricted as it was interfaced through the system's PLC, which was not accessible to this project. As a result, functionalities such as zeroing, calibration, and real-time data acquisition from the sensor could not be utilized. Hence, the sensor was unusable, and the internal force sensor of the UR5e had to be used instead.

Manual validation was attempted by physically pushing the ball and socket joint using an external handheld force gauge. However, this approach introduced several sources of error:

- Inconsistent angle of force application.
- Variable displacement, as hand-applied movements were not repeatable.
- Non-uniform force magnitude.

Moreover, this manual validation was not performed directly on the joystick mechanism, but rather on the ball and socket joint in isolation. As a result, the validation did not fully represent the actual conditions of joystick and thus could not reliably verify the forces experienced during end use scenarios.

Additionally, the theoretical force values calculated were based on idealised assumptions. These included perfect material interfaces, frictionless motion, and uniform contact, all of which diverge from real-world conditions. To improve the reliability of force validation in future work, several strategies could be employed:

- Gain access to the PLC system: This would allow full utilization of the high-accuracy force sensor currently mounted on the UR5e which would improve the accuracy of the force results calculated.
- Build a system for force validation: Rather than applying force manually, a linear actuator with force sensing capabilities could be installed to apply repeatable, controlled forces at the required positions and orientations.
- Direct force measurement at the joystick: Tactile force sensors can be installed directly on the hard stops of the joystick. For static interactions, the force exerted by the joystick onto the hard stop is equal and opposite to the force applied by the UR5e.

If the requirement shifts from measuring static forces to dynamic forces, an accelerometer can be mounted on top of the joystick. By measuring the joystick's acceleration and multiplying it by its known mass, the applied dynamic force can be estimated using Newton's Second Law. This method is suitable for capturing transient forces during the joystick's motion.

7.3 Making the stand for the socket joint

The stand was 3D printed to facilitate rapid prototyping and enable quick design iterations, particularly in anticipation of potential future modifications or attachments. However, 3D printing inherently produces poor-quality threaded holes, which can compromise mechanical stability. To address this, rivet nuts or metal thread inserts could be used to provide durable and secure clamping points.

Additionally, since the stand is located near the enclosure door, future attachments of bulky components to the UR5e could increase the risk of collision during the return motion, as the arm moves towards the door. A proposed solution is to incorporate a shallow recessed feature on the resting area of the socket component into the stand design. This would securely position the socket joint, allowing the UR5e to retract without dragging the socket joint along with it.

7.4 Other considerations

During testing, especially at the beginning, it was occasionally observed that the UR5e's movements exhibited inconsistencies, despite its high inherent accuracy. This phenomenon was later attributed to the UR5e's operating temperature, where it is designed to function optimally within a specified temperature range. A future improvement would be to have a standardised warm up routine movements that bring the UR5e to its operational temperature, before conducting the test movements.

8. Conclusion

This project successfully achieved its primary objectives: implementing the testing motions, verifying forces in four key directions, developing a user interface for the testing workflow, and designing and fabricating a stand for the socket joint. Multiple methodologies were explored and evaluated for each component, with the most suitable approaches selected based on performance, feasibility, and reliability. Although the chosen methods met the project goals, several limitations were identified, and corresponding improvements have been proposed for future iterations to enhance system performance and usability.

Furthermore, the project provided an in-depth introduction to automation arms, URScript programming, CAD modelling, and force verification methodologies. This project, given further refinement has the potential to speed up the testing of produced medical probes. Future work will focus on addressing the identified limitations, such as integrating more precise force sensors and refining calibration routines, to further improve both accuracy and efficiency. Future projects may also explore modifications to the mechanical testing process to enhance the accuracy of force validation.

9. References

- [1] "UR5 Basics Learn Module | UNSW Making," *www.making.unsw.edu.au*. <https://www.making.unsw.edu.au/learn/ur5-basics-learn-module/>
- [2] "The URScript Programming Language," 2018. Accessed: Apr. 19, 2025. [Online]. Available: <https://s3-eu-west-1.amazonaws.com/ur-support-site/32554/scriptManual-3.5.4.pdf>
- [3] K.-H. Choi, J.-K. Kim, H.-H. Shim, and Y.-K. Kong, "Gender-Based Differences in Actual Thumb Force Exertions at Various Target Force Levels," *Applied Sciences*, vol. 10, no. 1, p. 194, Dec. 2019, doi: <https://doi.org/10.3390/app10010194>.
- [4] "Force and torque values in tool coordinate system 44911," *Universal-robots.com*, 2025. <https://www.universal-robots.com/articles/ur/programming/get-force-and-torque-values-in-tool-coordinate-system/>
- [5] "Technical Specifications UR5e," *Universal-robots.com*, 2025. https://www.universal-robots.com/manuals/EN/HTML/SW10_6/Content/prod-usr-man/hardware/arm_e-Series/UR5e/H_g5_sections/appendix_g5/tech_spec_sheet.htm
- [6] Chapter 1 - Kinematics.pdf, "Chapter 1 - Kinematics.pdf," *Google Docs*, 2019. https://drive.google.com/file/d/1Fiuyuk5hrno3QryOHRjyLjRwM313IOKO/view?usp=drive_link

10. Appendix

Appendix A: Gantt Chart

No.	Start Date	End Date	Task	Hours used
Week 2				
1	1/20/2025	1/27/2025	Weekly Meeting	1:00
2	1/20/2025	1/27/2025	Meeting Minutes	0:30
3	1/20/2025	1/27/2025	Finish UR5E e-learning core, pro and optionally application track	2:00
4	1/20/2025	1/27/2025	Install URSIM	1:00
5	1/20/2025	1/27/2025	Develop a project timeline	0:30
Week 3				
6	1/27/2025	2/3/2025	Weekly Meeting	1:00
7	1/27/2025	2/3/2025	Meeting Minutes	0:30
8	1/27/2025	2/3/2025	Research on force/thumb control of joystick	2:00
Week 4				
9	2/3/2025	2/10/2025	Weekly Meeting	1:00
10	2/3/2025	2/10/2025	Meeting Minutes	0:30
11	2/3/2025	2/10/2025	Total Travelling time	2:00
12	2/3/2025	2/10/2025	Review specification of KPI's of TDS (Email Contents)	1:00
13	2/3/2025	2/10/2025	Discuss with TDS the current progress of product and issues faced.	1:30
14	2/3/2025	2/10/2025	Discuss with TDS the potential solutions and general direction of the project.	1:00
Week 5				
15	2/10/2025	2/17/2025	Weekly Meeting	1:00
16	2/10/2025	2/17/2025	Meeting Minutes	0:30
17	2/10/2025	2/17/2025	Total Travelling Time	2:00
18	2/10/2025	2/17/2025	Review legacy UR5E code provided by TDS. (Without ball and socket joint)	2:00
19	2/10/2025	2/17/2025	Formulate and discuss potential improvements to the legacy code.	2:00
Week 6				
20	2/17/2025	2/24/2025	Weekly Meeting	1:00
21	2/17/2025	2/24/2025	Meeting Minutes	0:30
22	2/17/2025	2/24/2025	Total Travelling Time	2:00
23	2/17/2025	2/24/2025	Review new design and workflow (Ball and socket joint).	2:00
24	2/17/2025	2/24/2025	Utilise the workflow to create a new test case on the UR5E (At the site).	2:00
Recess Week				
25	2/25/2025	3/3/2025	Weekly Meeting	1:00
26	2/25/2025	3/3/2025	Meeting Minutes	0:30
27	2/25/2025	3/3/2025	Total Travelling time	2:00
28	2/25/2025	3/3/2025	Review of customer evaluation.	1:00
29	2/25/2025	3/3/2025	Familiarisation with the URScript of new workflow	1:00
30	2/25/2025	3/3/2025	Identify areas for improvement in the current URScript.	3:00

31	2/25/2025	3/3/2025	Implement and verify improvements identified	4:00
			Week 7	
32	2/24/2025	3/24/2025	Weekly Meeting	1:00
33	2/24/2025	3/24/2025	Meeting Minutes	0:30
34	2/24/2025	3/24/2025	Travelling Time	2:00
35	2/24/2025	3/24/2025	Implement further workflow updates, TCP updates	3:00
36	2/24/2025	3/24/2025	3D Print the Step files for TDS design	1:00
37	2/24/2025	3/24/2025	Force Calculation Evaluations	2:30
38	2/24/2025	3/24/2025	Meeting with industrial partners	1:00
39	2/24/2025	3/24/2025	Discuss requirements for the stand	0:30
			Week 8	
40	3/24/2025	4/14/2025	Weekly Meeting	1:00
41	3/24/2025	4/14/2025	Meeting and site Minutes	1:30
42	3/24/2025	4/14/2025	Force Evaluations and verifications	6:00
43	3/24/2025	4/14/2025	Learning SolidWorks	3:00
44	3/24/2025	4/14/2025	Designing the stand	3:00
			Week 9 - 12	
45	3/24/2025	4/14/2025	Weekly Meeting	3:00
46	3/24/2025	4/14/2025	Meeting Minutes	1:30
47	3/24/2025	4/14/2025	Travelling Time	6:00
48	3/24/2025	4/14/2025	3D Print another step file for TDS design, and install it onto the UR5e	2:00
49	3/24/2025	4/14/2025	Improve CAD design for stand	3:00
50	3/24/2025	4/14/2025	Implement and test programming workflow for stand	3:00
51	3/24/2025	4/14/2025	Research and order parts required for stand	2:00
52	3/24/2025	4/14/2025	Final testing to ensure that KPI's are met.	8:00
53	3/24/2025	4/14/2025	Documentation writing	8:00
54	3/24/2025	4/14/2025	Wrap up and handover with TDS	4:00
			Week 13	
55	4/14/2025	4/18/2025	Weekly Meeting	1:00
56	4/14/2025	4/18/2025	Meeting Minutes	0:30
57	4/14/2025	4/18/2025	Evaluate the major challenges faced.	4:00
58	4/14/2025	4/18/2025	Note down improvements to be made for the next project iteration.	5:00
59	4/14/2025	4/18/2025	Finish UROP report	10:00

Total hours spent: 130 hours

Appendix B: Project Expenditure

	Description of items	Quantity	Amount
1	Star Knobs Thread Clamping	2	\$5.02
2	RS PRO Inductive Threaded Barrel Proximity Sensor, M12, 8 mm Detection, PNP NO, 10 → 30 V dc + Delivery	1	\$45.03
3	RS PRO Right Angle Female 3 way M12 to Untermated Sensor Actuator Cable, 2m	1	\$14.20

Total Amount Spent: \$64.25

Appendix C: Other CAD iterations of the stand for the socket joint

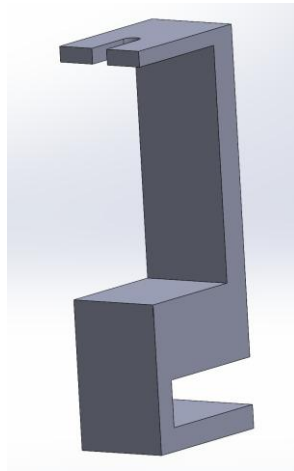


Figure 28: Preliminary Design of the stand

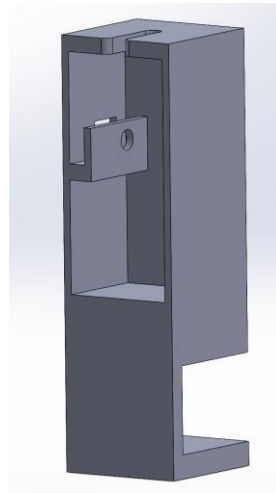


Figure 29: Intermediate iteration of stand, between the iterations of figure 24 and 25

Appendix D: Source code

For ease of viewing, the source code can be found [here](#).

Program

Variables Setup

Robot Program

```
set_tcp(p[0,0,0.22,0,0,0])
```

```
'Folder'
```

```
'Loop'
```

```
'Halt'
```

```
accel_var_g:=0.2
```

```
vel_var_g:=0.1
```

```
accel_var_cal:=0.02
```

```

vel_var_cal:=0.01
accel_var_op:=0.1
vel_var_op:=0.05
r_op:=0.005
Point_1:=pose_trans(Point_1, p[0,0,-0.1715,0,0,0])
Point_1_1:=pose_trans(Point_1, p[0,0,0.009,0,0,0])
MoveJ
    Start
    get_socket:='Get socket?'
    If get_socket= True
        MoveJ
        Stand
        movel(pose_trans(get_actual_tcp_pose(),
p[0,0,0.0015,0,0,0]),a=accel_var_g,v=vel_var_g)
        movel(pose_trans(get_actual_tcp_pose(),
0.044,0,0,0,0]),a=accel_var_g,v=vel_var_g)
        var_14:='Socket Gripped?'
        movel(pose_trans(get_actual_tcp_pose(), p[0,0.05,0,0,0,0]),a=accel_var_g,v=vel_var_g)
    MoveJ
    Start
    movel(pose_trans(Point_1, p[0,0,-0.030,0,0,0]),a=accel_var_g,v=vel_var_g)
    movel(Point_1,a=accel_var_g,v=vel_var_g)
    movel(pose_trans(Point_1, p[0,0,0.009,0,0,0]),a=accel_var_g,v=vel_var_g)
    var_1:='Calibration?'
    If var_1= True
        Folder
        Wait: 1.0
        zero_ftsensor()
        Wait: 1.0
        MoveL
        Direction: Tool X+
        Until (expression)
        Wait: 1.0

```



```

Limit_1:=get_actual_tcp_pose()
Force
    Wait Fx_tool≤-3 and Fy_tool≤-8
Wait: 1.0
Limit_2:=get_actual_tcp_pose()
movel(Limit_1,a=accel_var_cal,v=vel_var_cal)
Wait: 1.0
Force
    Wait Fx_tool≤-3 and Fy_tool≥8
Wait: 1.0
Limit_3:=get_actual_tcp_pose()
movel(pose_trans(Point_1, p[0,0,0.009,0,0,0]),a=accel_var_g,v=vel_var_g)
Wait: 1.0
zero_ftsensor()
Wait: 1.0
Direction: Tool X-
    Until (expression)
Wait: 1.0
Limit_4:=get_actual_tcp_pose()
Force
    Wait Fx_tool≥3 and Fy_tool≤-8
Wait: 1.0
Limit_5:=get_actual_tcp_pose()
movel(Limit_4,a=accel_var_cal,v=vel_var_cal)
Wait: 1.0
Force
    Wait Fx_tool≥3 and Fy_tool≥8
Wait: 1.0
Limit_6:=get_actual_tcp_pose()
movel(pose_trans(Point_1, p[0,0,0.009,0,0,0]),a=accel_var_g,v=vel_var_g)
Wait: 1.0
zero_ftsensor()

```

Wait: 1.0

Direction: Tool Y+

Until (expression)

Wait: 1.0

Limit_7:=get_actual_tcp_pose()

movel(pose_trans(Point_1, p[0,0,0.009,0,0,0]),a=accel_var_g,v=vel_var_g)

Wait: 1.0

zero_ftsensor()

Wait: 1.0

Direction: Tool Y-

Until (expression)

Wait: 1.0

Limit_8:=get_actual_tcp_pose()

movel(pose_trans(Point_1, p[0,0,0.009,0,0,0]),a=accel_var_g,v=vel_var_g)

VP 27

Point_1_1:=pose_trans(Point_1, p[0,0,0.009,0,0,0])

radius_2_7:=norm(Point_1_1[1]-Limit_7[1])

dist_1_2:=interpolate_pose(Limit_1, Limit_2, 1)

angle_2_7:=acos(dist_1_2[1]/norm(radius_2_7))

var_2:=pose_trans(Point_1_1, p[0,0,0,0,0,-angle_2_7/2])

var_3:=pose_trans(var_2,p[0,radius_2_7,0,0,0,0])

var_4:=pose_trans(var_3, p[0,0,0,0,0,angle_2_7/2])

VP 57

Point_1_1:=pose_trans(Point_1, p[0,0,0.009,0,0,0])

radius_5_7:=norm(Point_1_1[1]-Limit_7[1])

dist_4_5:=interpolate_pose(Limit_4, Limit_5, 1)

angle_5_7:=acos(dist_4_5[1]/norm(radius_5_7))

var_8:=pose_trans(Point_1_1, p[0,0,0,0,0,angle_5_7/2])

var_9:=pose_trans(var_8,p[0,radius_5_7,0,0,0,0])

var_10:=pose_trans(var_9, p[0,0,0,0,0,-angle_5_7/2])

VP 38

Point_1_1:=pose_trans(Point_1, p[0,0,0.009,0,0,0])

```

radius_3_8:=norm(Point_1_1[1]-Limit_8[1])
dist_1_3:=interpolate_pose(Limit_1, Limit_3, 1)
angle_3_8:=acos(norm(dist_1_3[1])/norm(radius_3_8))
var_5:=pose_trans(Point_1_1, p[0,0,0,0,0,angle_3_8/2])
var_6:=pose_trans(var_5,p[0,-radius_3_8,0,0,0,0])
var_7:=pose_trans(var_6, p[0,0,0,0,0,-angle_3_8/2])
VP 68
Point_1_1:=pose_trans(Point_1, p[0,0,0.009,0,0,0])
radius_3_8:=norm(Point_1_1[1]-Limit_8[1])
dist_4_6:=interpolate_pose(Limit_4, Limit_6, 1)
angle_6_8:=acos(norm(dist_4_6[1])/norm(radius_3_8))
var_11:=pose_trans(Point_1_1, p[0,0,0,0,0,-angle_6_8/2])
var_12:=pose_trans(var_11,p[0,-radius_3_8,0,0,0,0])
var_13:=pose_trans(var_12, p[0,0,0,0,0,angle_6_8/2])
TL:='TL'
TR:='TR'
TU:='TU'
TD:='TD'
Test_Force:='Test force'
num_loops:='Number of repetitons?'
Loop num_loops>0
If TL
Twist Left
movep(Limit_1,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_2,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_4,Limit_7,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movec(var_4,Limit_2,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_3,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_7,Limit_8,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movec(var_7,Limit_3,a=accel_var_op,v=vel_var_op,r=r_op)

```

movep(Limit_1,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)

If TR

Twist Right

movep(Limit_4,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Limit_5,a=accel_var_op,v=vel_var_op,r=r_op)

movec(var_10,Limit_7,a=accel_var_op,v=vel_var_op,r=0)

stopj(d2r(80))

movec(var_10,Limit_5,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Limit_6,a=accel_var_op,v=vel_var_op,r=r_op)

movec(var_13,Limit_8,a=accel_var_op,v=vel_var_op,r=0)

stopj(d2r(80))

movec(var_13,Limit_6,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Limit_4,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)

If TU

Twist Up

movep(Limit_7,a=accel_var_op,v=vel_var_op,r=r_op)

movec(var_10,Limit_5,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Limit_4,a=accel_var_op,v=vel_var_op,r=0)

stopj(d2r(80))

movep(Limit_5,a=accel_var_op,v=vel_var_op,r=r_op)

movec(var_10,Limit_7,a=accel_var_op,v=vel_var_op,r=r_op)

movec(var_4,Limit_2,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Limit_1,a=accel_var_op,v=vel_var_op,r=0)

stopj(d2r(80))

movep(Limit_2,a=accel_var_op,v=vel_var_op,r=r_op)

movec(var_4,Limit_7,a=accel_var_op,v=vel_var_op,r=r_op)

movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)

If TD

Twist Down

movep(Limit_8,a=accel_var_op,v=vel_var_op,r=r_op)

```

movec(var_13,Limit_6,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_4,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movep(Limit_6,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_13,Limit_8,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_7,Limit_3,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Limit_1,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movep(Limit_3,a=accel_var_op,v=vel_var_op,r=r_op)
movec(var_7,Limit_8,a=accel_var_op,v=vel_var_op,r=r_op)
movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)
If Test_Force
  TestUp:='TestUp'
  If TestUp
    Test up
      movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)
      stopj(d2r(80))
      movep(Limit_7,a=accel_var_op,v=vel_var_op,r=r_op)
      stopj(d2r(80))
      Wait: 1.0
      ForcePopUps
        Popup
        Popup
        Popup
        Popup
        Popup
        Popup
        Popup
        Popup
      TestLeft:='TestLeft'
      If TestLeft
        Test Left

```

```

movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)
stopj(d2r(80))
movep(Limit_1,a=accel_var_op,v=vel_var_op,r=r_op)
stopj(d2r(80))
Wait: 1.0
ForcePopUps
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup
  Popup
TestRight:='TestRight'
If TestRight
  Test Right
    movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)
    stopj(d2r(80))
    movep(Limit_4,a=accel_var_op,v=vel_var_op,r=r_op)
    stopj(d2r(80))
    Wait: 1.0
    ForcePopUps
      Popup
      Popup
      Popup
      Popup
      Popup
      Popup
      Popup
      Popup
      Popup
TestDown:='TestDown'

```


If TestDown

Test Down

movep(Point_1_1,a=accel_var_op,v=vel_var_op,r=0)

stopj(d2r(80))

movep(Limit_8,a=accel_var_op,v=vel_var_op,r=r_op)

stopj(d2r(80))

Wait: 1.0

ForcePopUps

Popup

Popup

Popup

Popup

Popup

Popup

Popup

Popup

num_loops:=num_loops-1

MoveJ

Start

return_socket:='Return socket?'

If return_socket $\neq$ True

MoveJ

Stand

move(pose_trans(get_actual_tcp_pose(),
p[0,0,0.0013,0,0,0]),a=accel_var_g,v=vel_var_g)

move(pose_trans(get_actual_tcp_pose(), p[0,0.04,0,0,0,0]),a=accel_var_g,v=vel_var_g)

var_14:='Socket Released?'

move(pose_trans(get_actual_tcp_pose(), p[0,0.05,0,0,0,0]),a=accel_var_g,v=vel_var_g)

Thread_1

extension_length:=0.092

joystick_length:=0.025

F_base:=get_tcp_force()

```

Fx_base:=F_base[0]
Fy_base:=F_base[1]
Fz_base:=F_base[2]
tcp:=get_actual_tcp_pose()
tcp_base:=p[tcp[0],tcp[1],tcp[2],0,0,0]
F_tool:=pose_trans(pose_inv(tcp),
p[Fx_base,Fy_base,Fz_base,0,0,0]))
Fx_tool:=F_tool[0]
Fy_tool:=F_tool[1]
Fz_tool:=F_tool[2]
Fx_Joystick:=F_tool[0]*(extension_length/joystick_length)
Fy_Joystick:=F_tool[1]*(extension_length/joystick_length)
Fz_Joystick:=F_tool[2]*(extension_length/joystick_length)
XY_Trans:=point_dist(tcp, Point_1_1)
If XY_Trans>extension_length
    XY_Trans:=extension_length
Angle_Joystick:=r2d(asin(XY_Trans/extension_length))
sync()

```